- `TypeFilterAttribute`—Loads all the filter's dependencies from the DI container and uses them to create a new instance of the filter.
- `ServiceFilterAttribute`—Loads the filter *itself* from the DI container. The DI container takes care of the service lifetime and building the dependency graph. Unfortunately, you must also explicitly register your filter with the DI container:

```
builder.Services.AddTransient<EnsureRecipeExistsFilter>();
```

TIP You can register your services with any lifetime you choose. If your service is registered as a singleton, you can consider setting the `IsReusable` flag, as described in the documentation: http://mng.bz/d1JD.

If you choose to use `ServiceFilterAttribute` instead of `TypeFilterAttribute`, and register the `EnsureRecipeExistsFilter` as a service in the DI container, you can apply the `ServiceFilterAttribute` directly to an action method:

```
[ServiceFilter(typeof(EnsureRecipeExistsFilter))]
public IActionResult Index() => Ok();
```

Whether you choose to use `TypeFilterAttribute` or `ServiceFilterAttribute` is somewhat a matter of preference, and you can always implement a custom `IFilterFactory` if you need to. The key takeaway is that you can now use DI in your filters. If you don't need to use DI for a filter, implement it as an attribute directly, for simplicity.